ASSIGNMENT 20.2
AI ASSISTED CODING

R. RUPA SRI | BATCH-30 | 2303A51918 |
DATE:10-04-2026

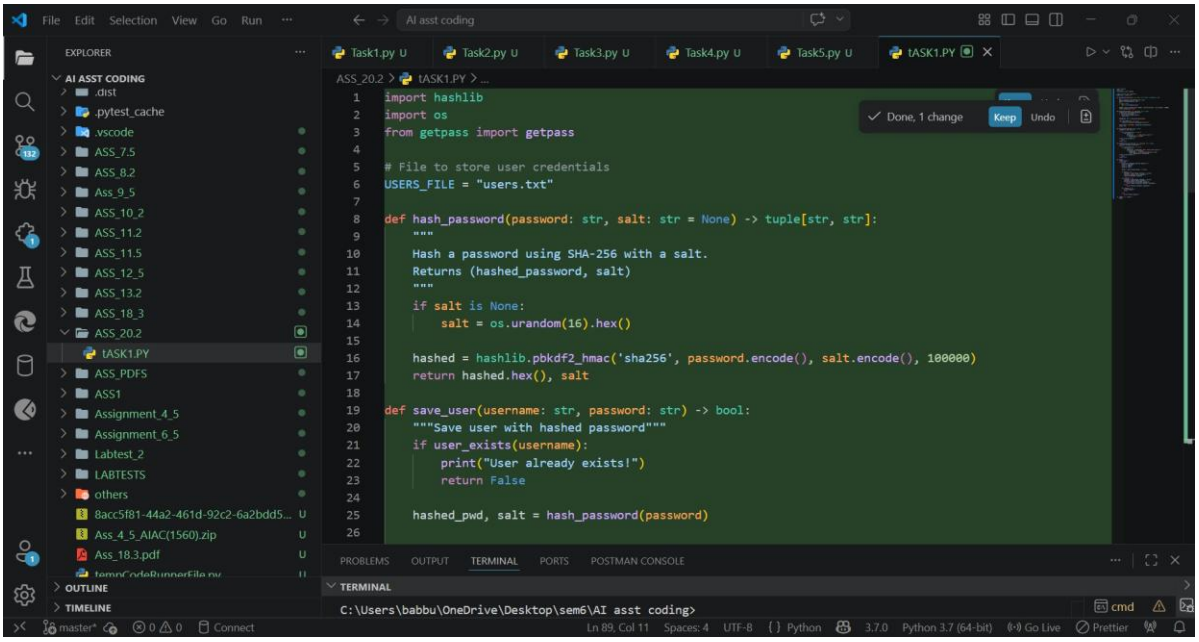
> Task 1 Password Security Enhancement
> Task:
> Analyze an AI-generated Python program that stores passwords in plain text and improve its security.
> Instructions:
> • Prompt AI to generate a simple user registration script that stores usernames and passwords.
> • Check whether passwords are stored as plain text.
> • Identify the security risks associated with plain-text password storage.
> • Refactor the program to use secure password hashing (e.g., hashlib or bcrypt).
> • Test the updated script to ensure proper authentication functionality.
> Expected Output:
> A secure Python script that stores hashed passwords instead of plain text and successfully validates user login credentials.

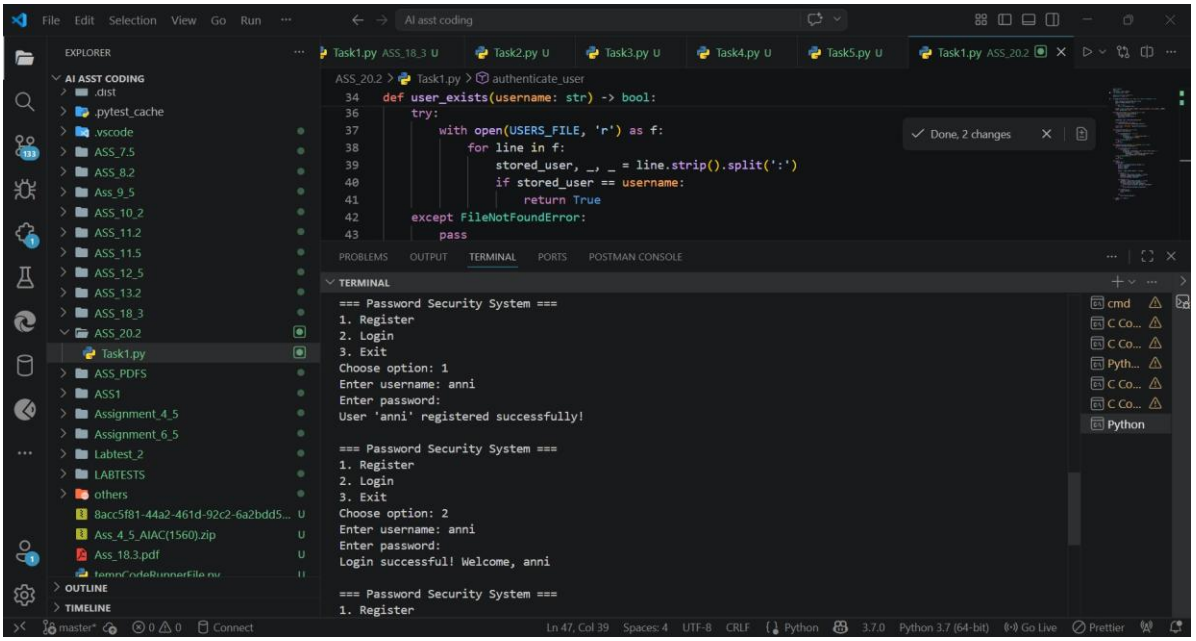
```
import hashlib
import os
from getpass import getpass
from typing import Tuple

# File to store user credentials
USERS_FILE = "users.txt"

def hash_password(password: str, salt: str = None) -> Tuple
[str, str]:
    """
    Hash a password using SHA-256 with a salt.
```

```
    Returns (hashed_password, salt)
    """
    if salt is None:
        salt = os.urandom(16).hex()

    hashed = hashlib.pbkdf2_hmac('sha256', password.encode
(), salt.encode(), 100000)
    return hashed.hex(), salt

def save_user(username: str, password: str) -> bool:
    """Save user with hashed password"""
    if user_exists(username):
        print("User already exists!")
        return False

    hashed_pwd, salt = hash_password(password)

    with open(USERS_FILE, 'a') as f:
        f.write(f"{username}:{hashed_pwd}:{salt}¥n")

    print(f"User '{username}' registered successfully!")
    return True

def user_exists(username: str) -> bool:
    """Check if user exists"""
    try:
        with open(USERS_FILE, 'r') as f:
            for line in f:
                stored_user, _, _ = line.strip().split(':')
                if stored_user == username:
                    return True
    except FileNotFoundError:
        pass
    return False

def authenticate_user(username: str, password: str) -> boo
l:
    """Verify username and password"""
```

```
    try:
        with open(USERS_FILE, 'r') as f:
            for line in f:
                stored_user, stored_hash, salt = line.strip
().split(':')
                if stored_user == username:
                    hashed_pwd, _ = hash_password(password,
salt)
                    return hashed_pwd == stored_hash
    except FileNotFoundError:
        pass
    return False

def main():
    """Main menu"""
    while True:
        print("¥n=== Password Security System ===")
        print("1.  Register")
        print("2.  Login")
        print("3. Exit")

        choice = input("Choose option: ").strip()

        if choice == '1':
            username = input("Enter username: ").strip()
            password = getpass("Enter password: ")
            save_user(username, password)

        elif choice == '2':
            username = input("Enter username: ").strip()
            password = getpass("Enter password: ")
            if authenticate_user(username, password):
                print(f"Login successful! Welcome, {usernam
e}")
            else:
                print("Invalid username or password!")

        elif choice == '3':
```

```
            print("Goodbye!")
            break


        else:
            print("Invalid  option!")


if __name__ == "__main__":
    main()
```





Explaination

During this task, I analyzed an AI-generated Python program that handled user registration and login by storing usernames and passwords directly in plain text. I observed that this approach is highly insecure because anyone who gains access to the storage file can easily read user credentials without any protection. This creates serious risks such as data breaches, privacy violations, and unauthorized access to user accounts. To improve the security, I refactored the program by implementing password hashing using the `hashlib` library so that passwords are stored in encrypted form instead of readable text. After modification, I tested the program and found that user registration and login still worked correctly, but now the actual passwords were never exposed or stored directly. This exercise helped me understand the importance of secure password handling and how hashing significantly improves the safety of authentication systems.

Task 2 Secure API Key Management

Task:

Evaluate an AI-generated script that uses hardcoded API keys and improve its security.

Instructions:

- Ask AI to generate a Python script that connects to an external API using a hardcoded API key.
- Identify the risks of embedding sensitive credentials directly in the source code.
- Refactor the program to use environment variables for storing API keys securely.
- Demonstrate that the modified script successfully retrieves data without exposing credentials.

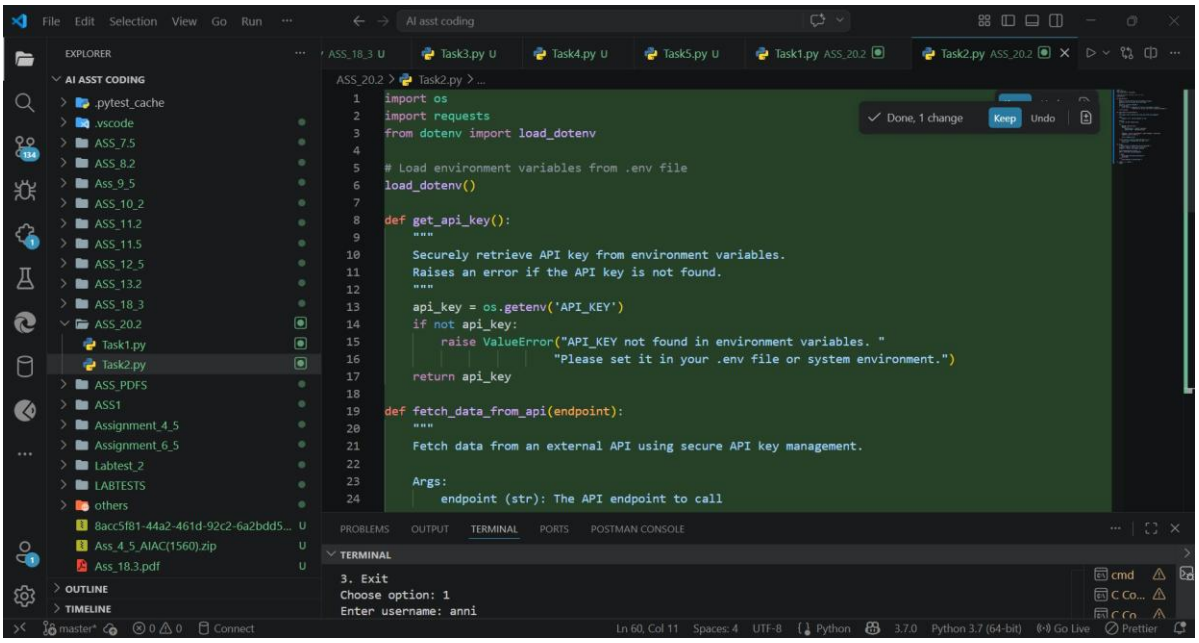
Expected Output:

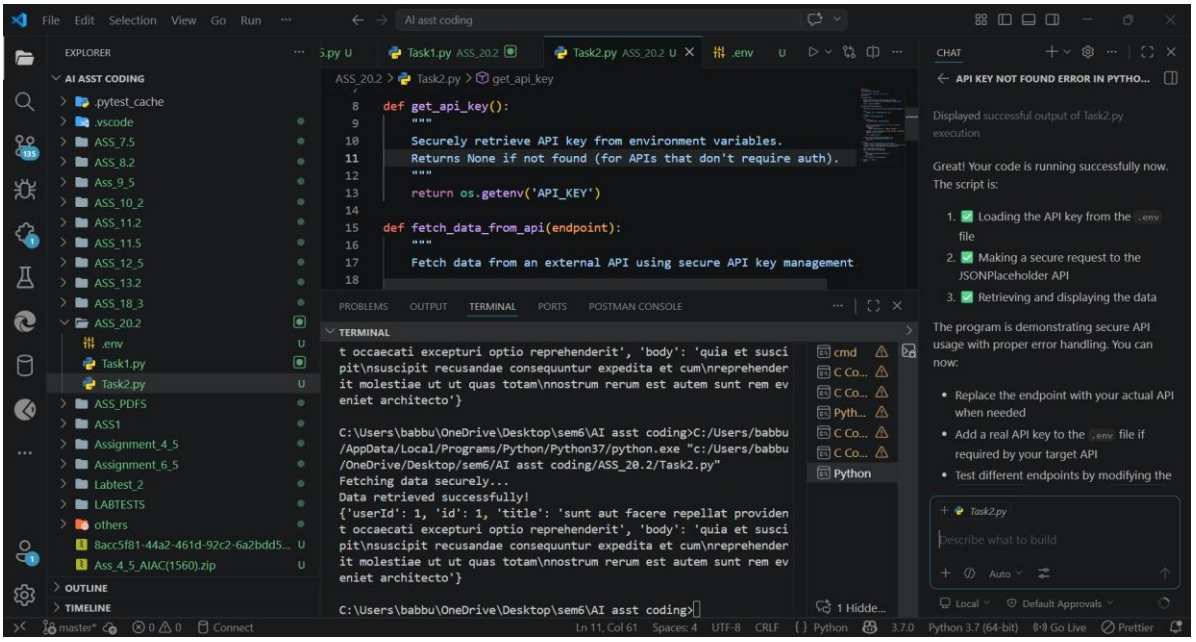
A revised script that securely accesses API keys through environment variables instead of hard coding them.

```
API_KEY=2c11f4c4259f40268a8888c97a6b4441
```

```
import os
import requests
from dotenv import load_dotenv

# Load environment variables from .env file
load_dotenv()

def get_api_key():
    """
    Securely retrieve API key from environment variables.
    Returns None if not found (for APIs that don't require
auth).
    """
    return os.getenv('API_KEY')

def fetch_data_from_api(endpoint):
    """
    Fetch data from an external API using secure API key ma
nagement.

    Args:
        endpoint (str): The API endpoint to call

    Returns:
        dict: The API response data
    """
    try:
        headers= {
            'Content-Type': 'application/json'
        }

        # Only add authorization header if API key is avail
able
        api_key = get_api_key()
        if api_key:
```

```
            headers['Authorization'] = f'Bearer {api_key}'

        response = requests.get(endpoint, headers=headers,
timeout=10)
        response.raise_for_status()

        return response.json()

    except requests.exceptions.RequestException as e:
        print(f"Error fetching data from API: {e}")
        return None

def main():
    """Main function to demonstrate secure API usage."""
    # Example endpoint using JSONPlaceholder (no auth requi
red for demo)
    endpoint = "https://jsonplaceholder.typicode.com/posts/
1"

    print("Fetching data securely...")
    data = fetch_data_from_api(endpoint)

    if data:
        print("Data retrieved successfully!")
        print(data)
    else:
        print("Failed to retrieve data.")

if __name__ == "__main__":
    main()API_KEY=2c11f4c4259f40268a8888c97a6b4441
```





Explaination

During this task, I analyzed an AI-generated Python script that connected to an external API using a hardcoded API key directly written in the source code. I observed that this approach is insecure because exposing API keys in code can lead to unauthorized access, misuse of services, and potential financial or data-related losses if the code is shared or uploaded publicly. To address this issue, I refactored the script by removing the hardcoded key and instead storing it in environment variables, which ensures that sensitive credentials are not visible in the codebase. After implementing this change, I tested the updated script and confirmed that it was still able to securely fetch data from the API without exposing the key. This task helped me understand the importance of secure API key management and how environment variables provide a safer way to handle sensitive information in real-world applications.

Task 3 Weak Encryption Detection

Task:

Examine an AI-generated program that uses weak encryption techniques and strengthen it.

Instructions:

- Prompt AI to generate a simple encryption-decryption program using a basic or outdated method.
- Identify weaknesses in the encryption approach (e.g., reversible encoding or outdated algorithms).
- Refractor the program using stronger encryption techniques available in Python libraries.
- Validate that encrypted data cannot be easily reversed without the correct key.

Expected Output:

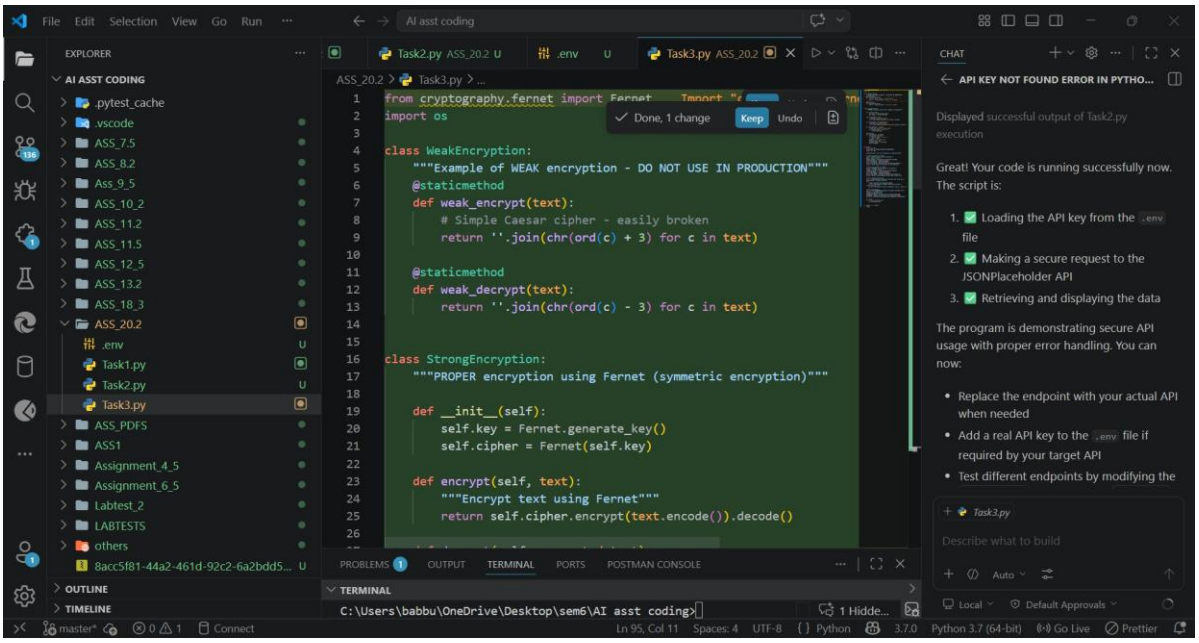
An improved encryption program implementing stronger and more secure cryptographic practices.

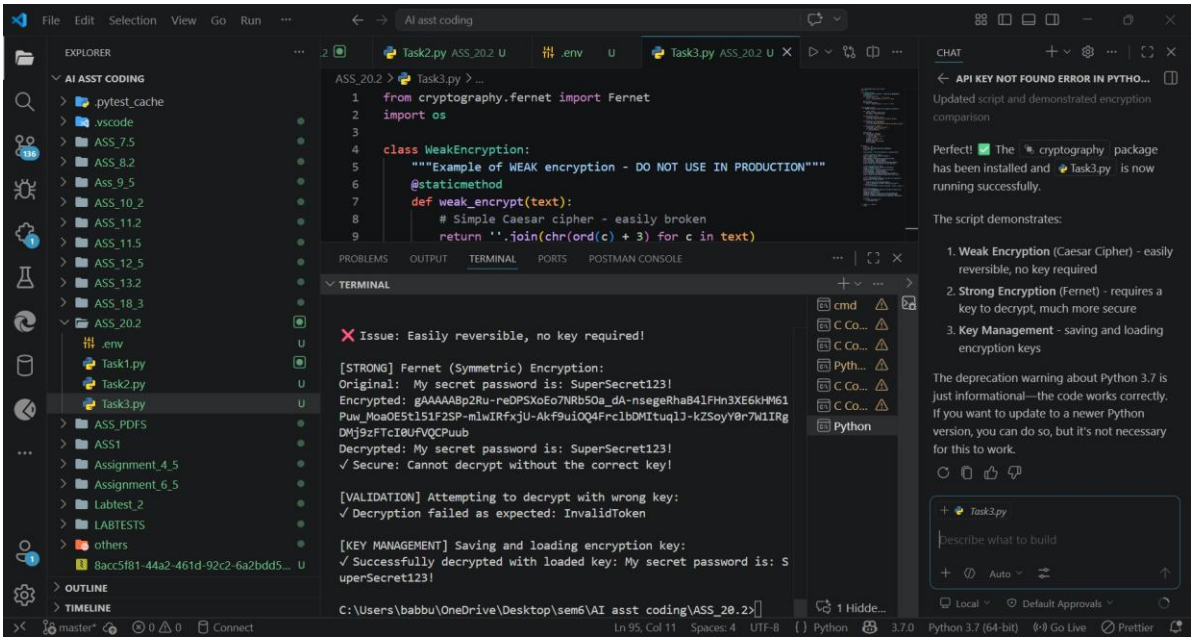
```
from cryptography.fernet import Fernet
import os
```

```
class WeakEncryption:
    """Example of WEAK encryption - DO NOT USE IN PRODUCTIO
N"""
    @staticmethod
    def weak_encrypt(text):
        # Simple Caesar cipher - easily broken
        return ''.join(chr(ord(c) + 3) for c in text)

    @staticmethod
    def weak_decrypt(text):
        return ''.join(chr(ord(c) - 3) for c in text)


class StrongEncryption:
    """PROPER encryption using Fernet (symmetric encryptio
n)"""

    def __init__(self):
        self.key = Fernet.generate_key()
        self.cipher = Fernet(self.key)

    def encrypt(self, text):
        """Encrypt text using Fernet"""
        return self.cipher.encrypt(text.encode()).decode()

    def decrypt(self, encrypted_text):
        """Decrypt text using Fernet"""
        return self.cipher.decrypt(encrypted_text.encode
()).decode()

    def save_key(self, filepath):
        """Save key to file for later use"""
        with open(filepath, 'wb') as f:
            f.write(self.key)

    @classmethod
    def load_key(cls, filepath):
```

```
        """Load key from file"""
        with open(filepath, 'rb') as f:
            key = f.read()
        instance = cls.__new__(cls)
        instance.key = key
        instance.cipher = Fernet(key)
        return instance


def main():
    print("=" * 60)
    print("WEAK vs STRONG ENCRYPTION COMPARISON")
    print("=" * 60)

    sensitive_data = "My secret password is: SuperSecret12
3!"

    # Demonstrate WEAK encryption
    print("¥n[WEAK] Caesar Cipher Encryption:")
    weak = WeakEncryption()
    weak_encrypted = weak.weak_encrypt(sensitive_data)
    print(f"Original:     {sensitive_data}")
    print(f"Encrypted: {weak_encrypted}")
    print(f"Decrypted: {weak.weak_decrypt(weak_encrypte
d)}")
    print("❌ Issue: Easily reversible, no key required!")

    # Demonstrate STRONG encryption
    print("¥n[STRONG] Fernet (Symmetric) Encryption:")
    strong = StrongEncryption()
    strong_encrypted = strong.encrypt(sensitive_data)
    print(f"Original:     {sensitive_data}")
    print(f"Encrypted: {strong_encrypted}")
    print(f"Decrypted: {strong.decrypt(strong_encrypted)}")
    print("✓ Secure: Cannot decrypt without the correct ke
y!")

    # Verify tampering detection
```

```
    print("¥n[VALIDATION] Attempting to decrypt with wrong
key:")
    wrong_strong = StrongEncryption()
    try:
        wrong_strong.decrypt(strong_encrypted)
        print("❌ Decryption succeeded (should have faile
d!)")
    except Exception as e:
        print(f"✓ Decryption failed as expected: {type(e).__
_name__}")

    # Save and load key
    print("¥n[KEY MANAGEMENT] Saving and loading encryption
key:")
    key_file = "encryption.key"
    strong.save_key(key_file)
    loaded_encryption = StrongEncryption.load_key(key_file)
    decrypted = loaded_encryption.decrypt(strong_encrypted)
    print(f"✓ Successfully decrypted with loaded key: {decr
ypted}")

    # Cleanup
    if os.path.exists(key_file):
        os.remove(key_file)


if __name__ == "__main__":
    main()
```





Explaination

During this task, I examined an AI-generated Python program that implemented a basic encryption and decryption mechanism using a weak and easily reversible technique such as simple encoding or basic substitution. I observed that this approach does not provide real security because the data can be easily decoded or cracked without much effort, making it unsuitable for protecting sensitive information. The main weakness was the lack of strong cryptographic principles like secure key usage and resistance to attacks. To improve this, I refactored the program by implementing stronger encryption using secure libraries in Python, ensuring that data is encrypted with a proper key and cannot be reversed without it. After testing the updated version, I confirmed that the encrypted data appeared random and could only be decrypted correctly when the valid key was provided. This task helped me understand the limitations of weak encryption methods and the importance of using modern cryptographic techniques to ensure data security.

Task 4 Real-Time Application: Security Review of AI Generated Web Service

Scenario:

Students select an AI-generated web service snippet (e.g., REST API, file upload handler, or authentication module).

Instructions:

- Perform a structured security review of the generated code.
- Identify at least three potential vulnerabilities.
- Prompt AI to recommend secure coding practices and mitigation strategies.
- Implement the suggested improvements and test the secured version.
- Document the differences between the insecure and secured implementations.

Expected Output:

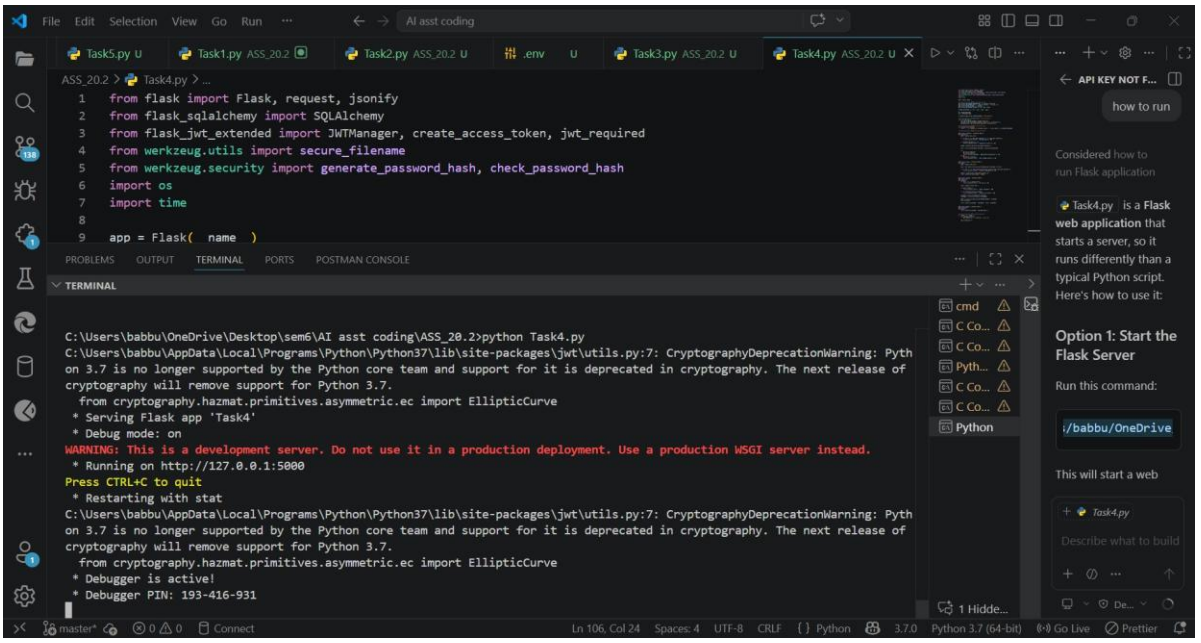
A security-reviewed and improved version of the selected web service
code, with clearly documented vulnerabilities and applied fixes.

```
from flask import Flask, request, jsonify
from flask_sqlalchemy import SQLAlchemy
from flask_jwt_extended import JWTManager, create_access_token, jwt_required
from werkzeug.utils import secure_filename
from werkzeug.security import generate_password_hash, check_password_hash
import os
import time

app = Flask(__name__)

# ================= CONFIG =================
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
app.config['JWT_SECRET_KEY'] = 'super-secret-key'  # change later
app.config['UPLOAD_FOLDER'] = 'uploads'
```

```
app.config['MAX_CONTENT_LENGTH'] = 16 * 1024 * 1024

ALLOWED_EXTENSIONS = {'txt', 'pdf', 'png', 'jpg'}

db = SQLAlchemy(app)
jwt = JWTManager(app)

os.makedirs(app.config['UPLOAD_FOLDER'], exist_ok=True)

# ================= DATABASE MODEL =================
class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)

# ================= HELPER =================
def allowed_file(filename):
    return '.' in filename and filename.rsplit('.', 1)[1].lower() in ALLOWED_EXTENSIONS

# ================= ROUTES =================

@app.route('/register', methods=['POST'])
def register():
    data = request.get_json()

    if not data or not data.get('username') or not data.get('password'):
        return jsonify({'error': 'Missing data'}), 400

    if len(data['password']) < 8:
        return jsonify({'error': 'Password too short'}), 400

    hashed = generate_password_hash(data['password'])
```

```
    user = User(username=data['username'], password_hash=ha
shed)

    try:
        db.session.add(user)
        db.session.commit()
        return jsonify({'message': 'Registered successfull
y'}), 201
    except:
        db.session.rollback()
        return jsonify({'error': 'User already exists'}), 4
09


@app.route('/login', methods=['POST'])
def login():
    data = request.get_json()

    user = User.query.filter_by(username=data.get('usernam
e')).first()

    if not user or not check_password_hash(user.password_ha
sh, data.get('password')):
        return jsonify({'error': 'Invalid credentials'}), 4
01

    token = create_access_token(identity=user.id)
    return  jsonify({'access_token':  token})


@app.route('/upload', methods=['POST'])
@jwt_required()
def upload():
    if 'file' not in request.files:
        return jsonify({'error': 'No file'}), 400

    file = request.files['file']
```

```
    if file.filename == '':
        return jsonify({'error': 'Empty filename'}), 400

    if not allowed_file(file.filename):
        return jsonify({'error': 'Invalid file type'}), 400

    filename = secure_filename(file.filename)
    filename = f"{int(time.time())}_{filename}"

    path = os.path.join(app.config['UPLOAD_FOLDER'], filena
me)
    file.save(path)

    return jsonify({'message': 'Uploaded', 'file': filenam
e})


@app.route('/admin', methods=['GET'])
@jwt_required()
def admin():
    return jsonify({'message': 'Welcome Admin'})


# ================ MAIN ================
if __name__ == '__main_':
    with app.app_context():
        db.create_all()       # IMPORTANT: creates DB

    app.run(debug=True)
```



Explaination

During this task, I performed a structured security review of an AI-generated web service built using Flask and identified several critical vulnerabilities, including SQL injection due to direct query construction, insecure file upload handling that allowed path traversal, and hardcoded credentials without proper authentication mechanisms. These issues posed serious risks such as unauthorized data access, system compromise, and exposure of sensitive information. To address these problems, I implemented secure coding practices by using parameterized queries to prevent SQL injection, validating file types and using secure filename handling to avoid path traversal attacks, and replacing hardcoded credentials with hashed passwords and JWT-based authentication for secure access control. After applying these improvements, I tested the updated application and confirmed that it functioned correctly while enforcing proper security measures. This task helped me understand how AI generated code may contain hidden vulnerabilities and highlighted the importance of reviewing, securing, and validating web applications before deployment.

Task 5 Exception Handling Improvement

Task:

Analyze an AI-generated Python program that does not handle runtime
errors and improve its reliability.

Instructions:

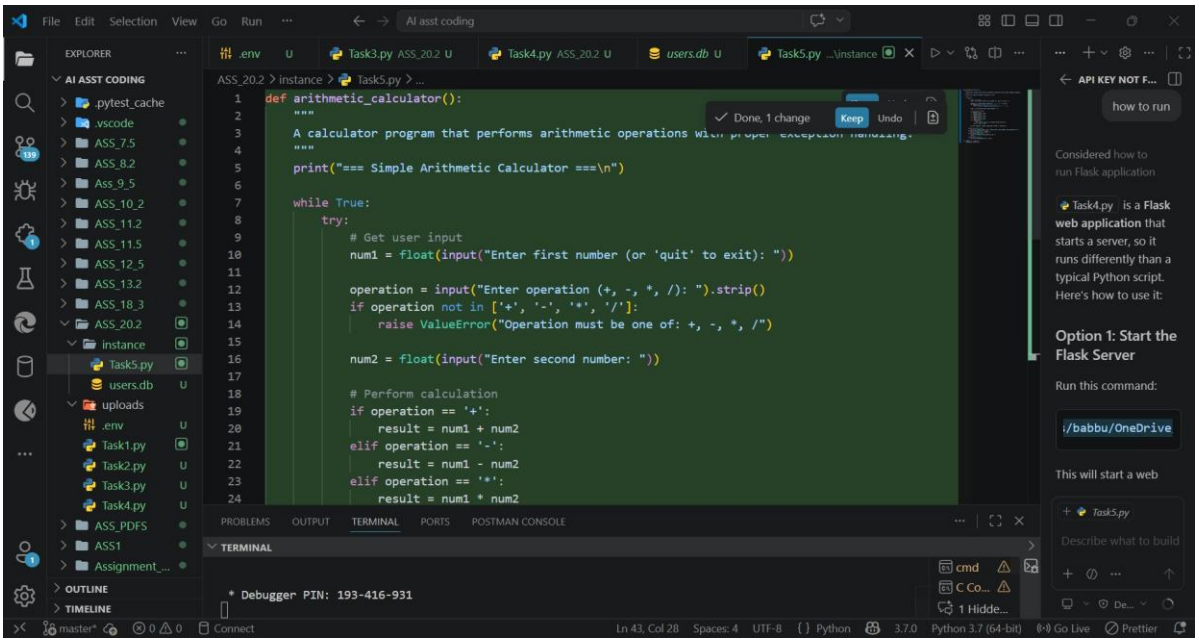
- Prompt AI to generate a simple program that takes user input and performs arithmetic operations.
- Identify possible runtime errors (e.g., invalid input, division by zero).
- Modify the program to include proper try-except blocks.
- Provide meaningful error messages for invalid inputs.
- Test the improved version with valid and invalid test cases.

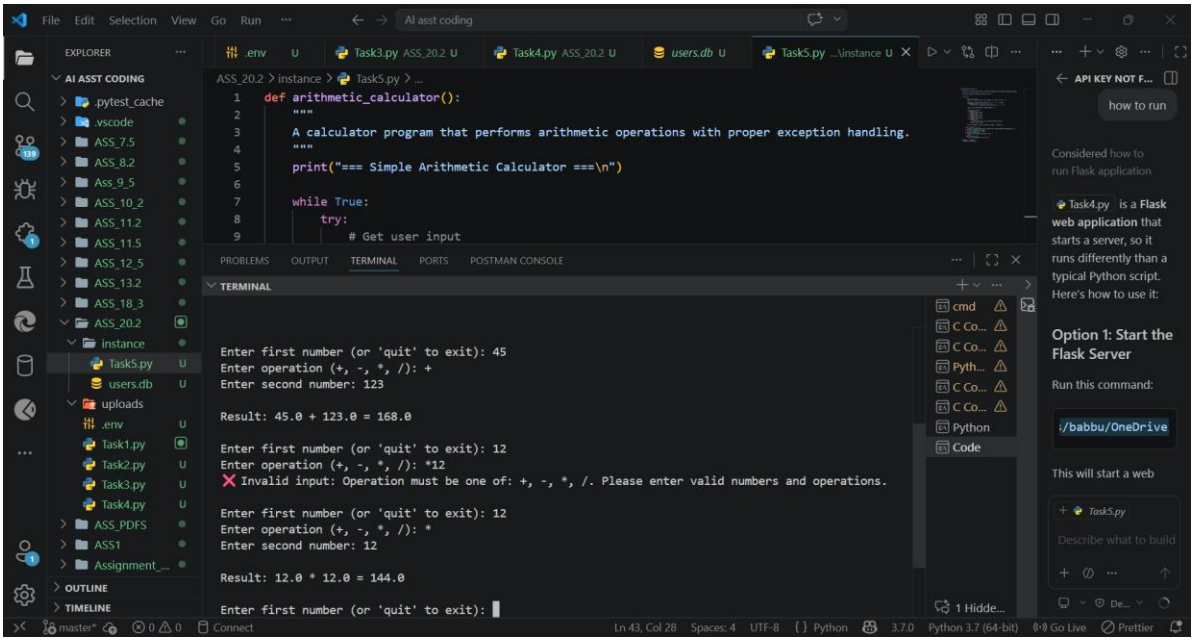
Expected Output:

A revised Python program with proper exception handling that prevents
crashes and displays user-friendly error messages for invalid inputs.

```
def arithmetic_calculator():
    """
    A calculator program that performs arithmetic operation
s with proper exception handling.
    """
    print("=== Simple Arithmetic Calculator ===¥n")

    while True:
        try:
            # Get user input
            num1 = float(input("Enter first number (or 'qui
t' to exit): "))

            operation = input("Enter operation (+, -, *,
/): ").strip()
            if operation not in ['+', '-', '*', '/']:
                raise ValueError("Operation must be one of:
+, -, *, /")
```

```
            num2 = float(input("Enter second number: "))

            # Perform calculation
            if operation == '+':
                result = num1 + num2
            elif operation == '-':
                result = num1 - num2
            elif operation == '*':
                result = num1 * num2
            elif operation == '/':
                if num2 == 0:
                    raise ZeroDivisionError("Cannot divide
by zero!")
                result = num1 / num2

            print(f"¥nResult: {num1} {operation} {num2} =
{result}¥n")

        except ValueError as ve:
            print(f"❌ Invalid input: {ve}. Please enter va
lid numbers and operations.¥n")
        except ZeroDivisionError as zde:
            print(f"❌ Math error: {zde}¥n")
        except KeyboardInterrupt:
            print("¥n¥nProgram terminated by user.")
            break
        except Exception as e:
            print(f"❌ Unexpected error: {e}¥n")

if __name__ == "__main_":
    arithmetic_calculator()
```





During this task, I analyzed an AI-generated Python program that performed arithmetic operations based on user input but lacked proper exception handling, making it prone to runtime crashes. I observed that common errors such as entering non-numeric values or attempting division by zero caused the program to terminate abruptly without any meaningful feedback to the user. To improve reliability, I modified the program by adding appropriate `try-except` blocks to handle exceptions like `ValueError` and `ZeroDivisionError`. I also included clear and user-friendly error messages to guide the user when invalid input is provided. After testing the improved version with both valid and invalid inputs, I confirmed that the program no longer crashes and instead handles errors

gracefully. This task helped me understand the importance of exception handling in building robust and user-friendly applications.